

TP1 – Tareas de FreeRTOS

Objetivos

Usar **Tareas** de **FreeRTOS**

Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Editar/compilar/depurar programas en **C** con **FreeRTOS**

Configurar el **IDE** para depurar en modo **semihosting**

Adoptar metodología orientada a la portabilidad/reúso de **código**

Identificar **Patrones de Diseño de Software**

Usar **Git** y **GitHub**

Generar/actualizar **repositorio** por línea de **comando**

Generar/actualizar **archivos** del tipo **README.md** (**Markdown**)

Usar **Gemini**

Analizar/explicar **código fuente**

Tareas

Cómo **FreeRTOS** asigna tiempo de procesamiento a cada **Tarea** en una aplicación

Cómo **FreeRTOS** elige qué **Tarea** debe ejecutarse en un momento dado

Cómo la prioridad relativa de cada **Tarea** afecta el comportamiento del sistema

Los estados en los que puede encontrarse una **Tarea**

Cómo implementar **Tareas**

Cómo crear una o más instancias de una **Tarea**

Cómo usar el parámetro de **Tarea**

Cómo cambiar la prioridad de una **Tarea** ya creada

Cómo eliminar una **Tarea**

Cómo implementar el procesamiento periódico mediante una **Tarea**

Cuándo se ejecutará la **Tarea IDLE** y cómo se puede utilizar

Actividades

1er encuentro

[TP1-01 – 4to Proyecto p/placa NUCLEO-F103RB con FreeRTOS](#)

[TP1-02 – 5to Proyecto p/placa NUCLEO-F103RB con FreeRTOS](#)

2do encuentro

[TP1-03 – 6to Proyecto p/placa NUCLEO-F103RB con FreeRTOS](#)

[TP1-04 – 7mo Proyecto p/placa NUCLEO-F103RB con FreeRTOS](#)

Resolución en grupos de **3** alumnos, requiere **presentación** por parte de un **responsable** del grupo

TP1 – Actividad 01 – 4to Proyecto p/placa NUCLEO-F103RB con FreeRTOS

Objetivos

Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

Usar **Git** y **GitHub**

Generar/Actualizar repositorio

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Usar **Gemini**

Analizar/Explicar código fuente

Paso 01: Crear un nuevo **repositorio** en **GitHub** para almacenar su **TP1**, pasos:

GitHub: Repositories => New =>

Repository name: **sotri-tp1_Cohorte-Grupo**

Description: # **CESE - Sistemas Operativos de Tiempo Real - Trabajo Práctico N°: 1 – Tareas de FreeRTOS**

Initialize this repository with:

Tildar **Add a README file**

=> **Create repository**

Cohorte-Grupo es: **2XCo202X-01** o **2XCo202X-02** ... etc. (el correspondiente al Curso y Grupo de cursada)

Paso 02: Invitar al **repositorio**, como **Colaboradores** a los **docentes** del curso, responsables de la aprobación de su **TP1**, pasos:

GitHub: Repositories => Settings => Collaborators => Tildar: **Add people** =>

en **Find people** => tipear: **"Username del Docente"**

Tildar **Add "Username del Docente"**

Una vez que los **docentes** de su curso, **hayan aceptado** la **invitación** a ser **Colaboradores**, podrá agregarlos como **Revisores**

PasPaso 03: **Asentar** los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **editar** el archivo **README.md** y **modificar** su contenido, pasos:

GitHub: **README.md** => **Edit this file** => **borrar** su contenido => **Copiar** y **Pegar** lo siguiente:

```
# CESE - Sistemas Operativos de Tiempo Real
## Trabajo Práctico N°: 1 - Tareas de FreeRTOS
### Cohorte-Grupo

### Responsable de la entrega:
| N° SIU | Apellidos, Nombres | Fecha | Deadline |
| :----- | :----- | :----- | :----- |
| XXXXXX | YYYY, ZZZ | | Semana 04 |
```

Actualizar el archivo **README.md** con los datos del responsable de la entrega (un alumno diferente para cada TP):

Cohorte-Grupo, **N° SIU**, **Apellidos** y **Nombres**, ...

Paso 04: Generar un nuevo **proyecto STM32**: descargar el archivo [sotri-tp1_01-application.zip](#) e importar el **proyecto STM32** que contiene: **sotri-tp1_01-application**.

Compilar el **proyecto STM32** (**STM32CubeIDE**) importado: **sotri-tp1_01-application**.

Paso 05: Asentar los detalles de las entregas en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), crear el archivo **sotri-tp1_01-application.md**.

Paso 06: Consultar a **Gemini** de **Google** respecto al código fuente del **proyecto STM32**, pasos:

Logear, en **Google** con tu cuenta de correo electrónico institucional, del dominio **@fi.uba.ar**.

Ingresar a **Gemini** => <https://gemini.google.com/app>, => **Nueva Conversación** => elegir modelo **Pro** => en **Preguntar a Gemini** =>

Copiar y Pegar la siguiente línea de **texto**:

Analizar y **explicar** (en español), el funcionamiento del código fuente contenido en los archivos adjuntos: **startup_stm32f103rbtx.s**, **main.c**, **stm32f1xx_it.c**, **FreeRTOSConfig.h** y **freertos.c**.

Indicar la evolución de las variables **SysTick** y **SystemCoreClock** al ejecutar dicho código fuente desde su inicio (**Reset_Handler**: de **startup_stm32f103rbtx.s**) hasta el loop principal de la aplicación (**while (1)** de **main.c**).

Indicar el comportamiento del programa al ejecutar dicho código fuente desde su inicio (**Reset_Handler**: de **startup_stm32f103rbtx.s**) hasta antes de llegar al loop principal de la aplicación (**while (1)** de **main.c**).

Indicar cómo y para qué **SysTick** y **Timer 1 (TIM2)** interactúan con **FreeRTOS**.

Indicar cómo y para qué el **Timer 2 (TIM2)** interactúa con la **HAL** del **proyecto STM32**.

Agregar los siguientes archivos (click en **+** Agregar archivos => **Subir archivos**):

```
...\sotri_workspace\sotri-tp1_01-application\Core Src\main.c
...\sotri_workspace\sotri-tp1_01-application\Core Src\stm32f1xx_it.c
...\sotri_workspace\sotri-tp1_01-application\Core Src\freertos.c
...\sotri_workspace\sotri-tp1_01-application\Core Incl\FreeRTOSConfig.h
...\sotri_workspace\sotri-tp1_01-application\Core Startup\startup_stm32f103rbtx.s
```

Leer y almacenar la respuesta en: **...\sotri_workspace\sotri-tp1_01-application\sotri-tp1_01-application.md**.

Paso 07: Depurar el nuevo **proyecto STM32**:

Confirmar mediante la depuración, el análisis, las explicaciones e indicaciones de la respuesta de **Gemini Pro** del **Paso TP1 – Actividad 01 – Paso 06**.

Paso 08: Consultar a **Gemini** de **Google** respecto al código fuente del **proyecto STM32**, pasos:

Ingresar a **Gemini** => <https://gemini.google.com/app>, => en **Preguntar a Gemini** =>

Copiar y Pegar la siguiente línea de **texto**.

Analizar y explicar (en español), el funcionamiento del código fuente contenido en los archivos adjuntos: **app.c**, **task_btn.c**, **task_led.c**, **task_led_interface.c**, y **freertos.c**.

Agregar los siguientes archivos (click en **+** Agregar archivos => **Subir archivos**):

```
...\sotri_workspace\sotri-tp1_01-application\app\src\app.c  
...\sotri_workspace\sotri-tp1_01-application\app\src\task_btn.c  
...\sotri_workspace\sotri-tp1_01-application\app\src\task_led.c  
...\sotri_workspace\sotri-tp1_01-application\app\src\task_led_interface.c  
...\sotri_workspace\sotri-tp1_01-application\app\src\freertos.c
```

Leer y almacenar la respuesta en: **...\sotri_workspace\sotri-tp1_01-application\sotri-tp1_01-application.md**.

Paso 09: Depurar el nuevo **proyecto STM32**.

Confirmar mediante la depuración, el análisis, las explicaciones e indicaciones de la respuesta de **Gemini Pro** del **Paso TP1 – Actividad 01 – Paso 08**.

Paso 10: Almacenar el **proyecto STM32**: **sotri-tp1_01-application** y el/los archivo/s de la actividad: **sotri-tp1_01-application.md**, en el **repositorio** creado en **GitHub**, en el **TP1 – Actividad 01 – Paso 01**.

TP1 – Actividad 02 – 5to Proyecto p/placa NUCLEO-F103RB con FreeRTOS

Objetivos

Usar **Tareas** de **FreeRTOS**

Crear/Eliminar Tareas (tiempo de procesamiento, secuencia de ejecución, estados, prioridades, instancias)

Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

Usar **Git** y **GitHub**

Generar/Actualizar repositorio por línea de **comando**

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Paso 01: Generar un nuevo **proyecto STM32**: **copiar** el archivo **sotri-tp1_01-application** y **pegar/renombrar** como **sotri-tp1_02-application**.

Renombrar el archivo **sotri-tp1_01-application.ioc** como **sotri-tp1_02-application.ioc**.

Borrar los archivos de compilación/programación/depuración (**sotri-tp1_01-application.launch** y **sotri-tp1_01-application.cfg**).

Compilar/Depurar el **proyecto STM32** (**STM32CubeIDE**) renombrado (**sotri-tp1_02-application**).

Generar los archivos de depuración (**sotri-tp1_02-application.launch** y **sotri-tp1_02-application.cfg**).

Paso 02: Asentar los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo **sotri-tp1_02-application.md**, **editar** y **modificar** su contenido respondiendo las siguientes preguntas (en base a su experiencia depurando el **proyecto STM32**):

- ¿Cómo **FreeRTOS** asigna tiempo de procesamiento a cada **Tarea** en una aplicación?
- ¿Cómo **FreeRTOS** elige qué **Tarea** debe ejecutarse en un momento dado?
- ¿Cómo la prioridad relativa de cada **Tarea** afecta el comportamiento del sistema?
- ¿Cuáles son los estados en los que puede encontrarse una **Tarea**?
- ¿Cómo implementar **Tareas**?
- ¿Cómo crear una o más instancias de una **Tarea**?
- ¿Cómo eliminar una **Tarea**?

Paso 03: Modificar las prioridades relativas asignadas a **task_btn** y **task_led**, **compilar/depurar/observar** comportamiento, **asentar** lo observado en el archivo **sotri-tp1_02-application.md** y **reestablecer** las prioridades relativas asignadas originales.

Paso 04: Crear tres instancias de **task_btn** para gestionar el mismo botón y que **task_led** elimine una de las instancias de **task_btn**, **compilar/depurar/observar** comportamiento y **asentar** lo observado en el archivo **sotri-tp1_02-application.md**.

Paso 05: Almacenar el **proyecto STM32**: **sotri-tp1_02-application** y el/los archivo/s de la actividad: **sotri-tp1_02-application.md**, en el **repositorio** creado en **GitHub**, en el **TP1 – Actividad 01 – Paso 01**.

TP1 – Actividad 03 – 6to Proyecto p/placa NUCLEO-F103RB con FreeRTOS

Objetivos

Usar **Tareas** de **FreeRTOS**

Crear/Eliminar Tareas (parámetros, cambio de, prioridades, instancias)

Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

Usar **Git** y **GitHub**

Generar/Actualizar repositorio por línea de **comando**

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Paso 01: Generar un nuevo **proyecto STM32**: **copiar** el archivo **sotri-tp1_01-application** y **pegar/renombrar** como **sotri-tp1_03-application**.

Renombrar el archivo **sotri-tp1_01-application.ioc** como **sotri-tp1_03-application.ioc**.

Borrar los archivos de compilación/programación/depuración (**sotri-tp1_01-application.launch** y **sotri-tp1_01-application.cfg**).

Compilar/Depurar el **proyecto STM32** (**STM32CubeIDE**) renombrado (**sotri-tp1_03-application**).

Generar los archivos de depuración (**sotri-tp1_03-application.launch** y **sotri-tp1_03-application.cfg**).

Paso 02: Asentar los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo **sotri-tp1_03-application.md**, **editar** y **modificar** su contenido respondiendo las siguientes preguntas (en base a su experiencia depurando el **proyecto STM32**):

¿Cómo usar el parámetro de **Tarea**?

¿Cómo cambiar la prioridad de una **Tarea** ya creada?

Paso 03: Modificar **task_btn** para poder gestionar dos botones diferentes usando el parámetro de **Tarea** para diferenciarlos.

Crear dos instancias de **task_btn** para gestionar botones diferentes, **compilar/depurar/observar** comportamiento y **asentar** lo observado en el archivo **sotri-tp1_03-application.md**.

Paso 04: Modificar la prioridad de **task_led** para asegurar que sea la primera tarea en ejecutar, para luego recuperar las prioridades relativas asignadas originales.

Crear dos instancias de **task_led** para gestionar led diferentes, **compilar/depurar/observar** comportamiento y **asentar** lo observado en el archivo **sotri-tp1_03-application.md**.

Paso 05: Almacenar el **proyecto STM32**: **sotri-tp1_03-application** y el/los archivo/s de la actividad: **sotri-tp1_03-application.md**, en el **repositorio** creado en **GitHub**, en el **TP1 – Actividad 01 – Paso 01**.

TP1 – Actividad 04 – 7mo Proyecto p/placa NUCLEO-F103RB con FreeRTOS

Objetivos

Usar **Tareas** de **FreeRTOS**

Crear/Eliminar Tareas (procesamiento periódico, IDLE)

Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

Usar **Git** y **GitHub**

Generar/Actualizar repositorio por línea de **comando**

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Paso 01: **Generar** un nuevo **proyecto STM32**: **copiar** el archivo **sotri-tp1_01-application** y **pegar/renombrar** como **sotri-tp1_04-application**.

Renombrar el archivo **sotri-tp1_01-application.ioc** como **sotri-tp1_04-application.ioc**.

Borrar los archivos de compilación/programación/depuración (**sotri-tp1_01-application.launch** y **sotri-tp1_01-application.cfg**).

Compilar/Depurar el **proyecto STM32** (**STM32CubeIDE**) renombrado (**sotri-tp1_04-application**).

Generar los archivos de depuración (**sotri-tp1_04-application.launch** y **sotri-tp1_04-application.cfg**).

Paso 02: **Asentar** los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo **sotri-tp1_04-application.md**, **editar** y **modificar** su contenido respondiendo las siguientes preguntas (en base a su experiencia depurando el **proyecto STM32**):

¿Cómo implementar el procesamiento periódico mediante una **Tarea**?

¿Cuándo se ejecutará la **Tarea IDLE** y cómo se puede utilizar?

Paso 03: **Modificar** **task_led** para implementar el procesamiento periódico de la misma, **compilar/depurar/observar** comportamiento y **asentar** lo observado en el archivo **sotri-tp1_04-application.md**.

Paso 04: **Modificar** **task_btn** para implementar el procesamiento periódico de la misma, **compilar/depurar/observar** comportamiento y **asentar** lo observado en el archivo **sotri-tp1_04-application.md**.

Paso 05: **Almacenar** el **proyecto STM32**: **sotri-tp1_04-application** y el/los archivo/s de la actividad: **sotri-tp1_04-application.md**, en el **repositorio** creado en **GitHub**, en el **TP1 – Actividad 01 – Paso 01**.

GitHub: Presionar: **Commit changes ...** (**Guardar** los cambios)

Tildar: **Create a new branch** for this commit and start a pull request

Presionar: **Propose changes**

Presionar: **Create pull request**, tipear el nombre del **branch** y **agregar** a los **docentes** de su curso, como **Reviewers** (para invitarlos a **revisar** sus **entregas** mediante un **pull request**).

Ver **cómo hacer el pull request** en este video: [Trabajo colaborativo con Github](#)

En la descripción del video se detalla el contenido (ver en [14:23](#) la sección "CÓMO SOLICITAR REVISIONES")
IMPORTANTE: Se recomienda que primero vean "[19:25](#) Solución a un problema muy habitual" y luego "[19:02](#)
Create pull request y Review requested"